

LLM-READY LOG ANALYSIS · AGENT RELIABILITY

대용량 로그 분석 & Judge Agent

로그를 템플릿으로 압축해 LLM에 넣고, 에이전트의 drift를 telemetry로 관측·검증한다

Drain3

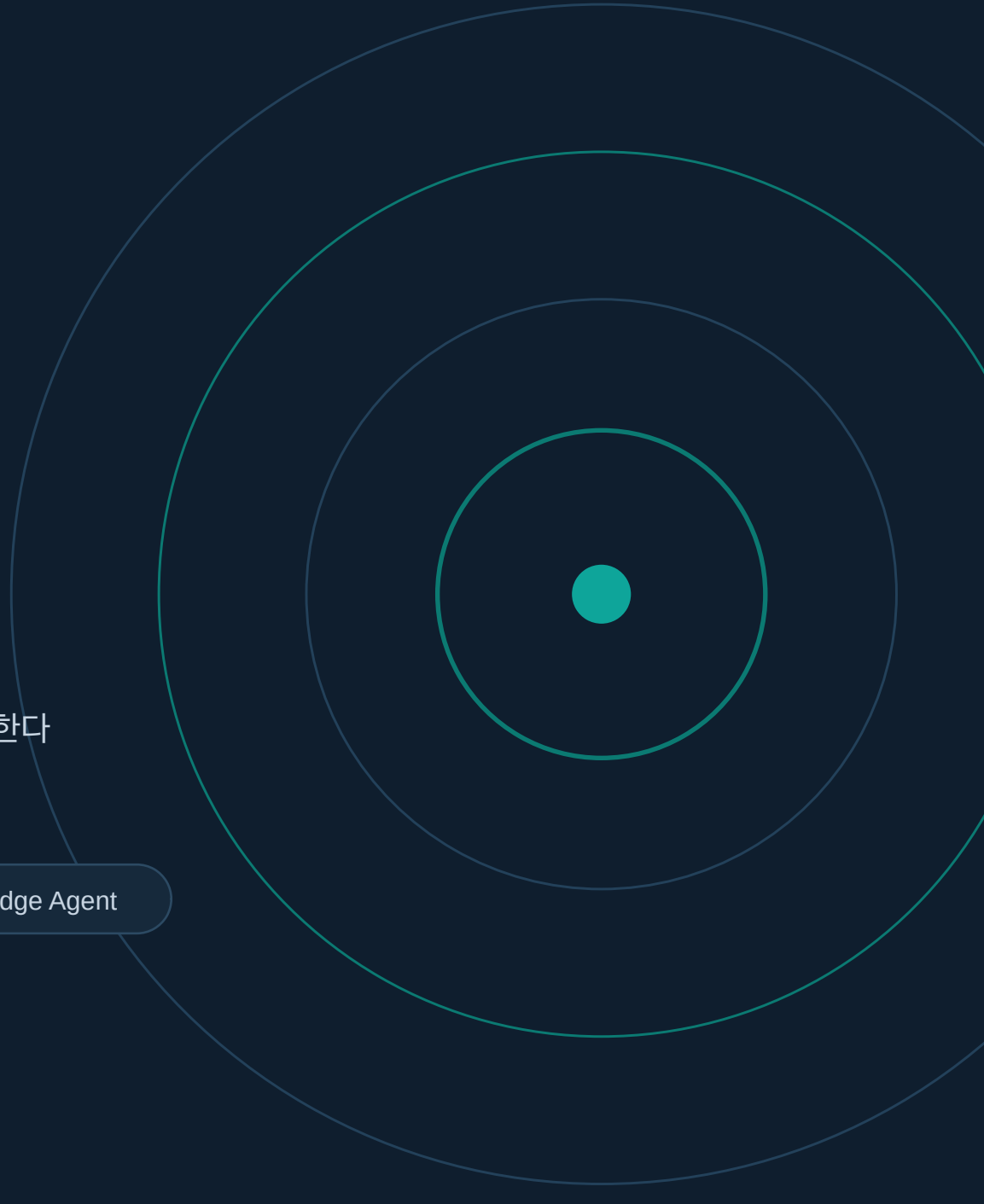
pandas

LangGraph

Drift Telemetry

Judge Agent

2026.05.29



발표 구성



PART 1

대용량 로그 분석

Drain3 + pandas

로그를 소수의 템플릿으로 압축하고, 원본-템플릿 매핑을 SQL식으로 질의해 LLM 분석에 사용한다.

문제

핵심 아이디어

압축 · 매핑 · 질의

워크플로우



PART 2

Judge Agent & Drift Telemetry

Context Engineering · LangGraph

운영 가능한 AI 에이전트를 위한 컨텍스트 설계, LangGraph 구조화, Drift 관측과 Judge Agent 검증.

Agent Drift

Context Engineering

LangGraph · Telemetry

Judge Agent

로그를 그대로 LLM에 넣을 수 없다

운영 환경은 매일 수백만 줄의 로그를 쏟아냅니다. 가공 없이 LLM으로 분석하려 하면 세 가지 벽에 부딪칩니다.



01

컨텍스트 한계

수십만 줄 로그는 모델의 토큰 한도를 즉시 초과 — 분석 자체가 불가능합니다.



02

토큰 = 비용

거의 동일한 로그가 반복되는데, 그 중복분예까지 토큰 비용을 그대로 지불합니다.



03

신호 희석

동일 패턴의 반복이 정작 중요한 이상 징후를 묻어 분석 정확도를 떨어뜨립니다.

로그는 사실 소수의 반복 패턴이다

대부분의 로그는 변하는 값(사용자·IP·시간)만 다를 뿐 구조는 동일합니다.

변하는 값을 `<*>`로 치환하면 중복이 사라지고, 수만 줄이 소수의 템플릿으로 압축됩니다.

개념 예시 — 압축비

10,000 줄 → **50 템플릿**

≈ 99% 중복 제거 → 토큰 대폭 절감

Login failed for user **guest** from IP **192.168.1.10**

Login failed for user **admin** from IP **10.0.0.5**

Login failed for user **kim** from IP **172.16.0.9**

... 9,997 줄 더



1 TEMPLATE

Login failed for user `<*>` from IP `<*>`

압축 → 매핑 → 질의

Drain3와 pandas만으로 대용량 로그를 LLM이 다룰 수 있는 형태로 바꾸고, SQL처럼 분석합니다.



STEP 01

압축

Drain3 템플릿화

반복 로그를 템플릿(<*>)으로 압축해
LLM 입력 토큰을 절감



STEP 02

매핑

원본-템플릿 DataFrame

template_id로 압축 템플릿과
원본 로그를 연결



STEP 03

질의

SQL식 분석

WHERE·LIKE·BETWEEN·GROUP BY
처럼 로그를 조회

Drain3 템플릿화로 로그를 압축한다

Drain3 TemplateMiner가 메시지 패턴을 자동 학습해, 변하는 값을 <*>로 치환한 템플릿으로 클러스터링합니다.

Login failed for user guest from IP 192.168.1.10



Login failed for user <*> from IP <*>

동일 패턴의 로그를 하나의 클러스터(template_id)로 묶어 중복을 제거합니다.

압축 강도 조절 — 주요 파라미터

파라미터	기본값	설명
sim_th	0.4	유사도 임계값 (낮을수록 더 압축)
depth	4	Drain 트리 깊이
max_children	100	노드당 최대 자식 수
parametrize_numeric_tokens	True	숫자 토큰을 <*>로 대체

핵심 함수

build_template_miner()

Drain3 인스턴스 생성

extract_templates_from_entries()

로그 목록 일괄 압축

get_all_clusters()

학습된 전체 템플릿 조회

build_cluster_summary_dataframe()

템플릿별 로그 건수 요약

토큰 사용량이 급감한다

원본 로그 대신 템플릿 + 건수만 LLM에 전달하면, 분석 비용과 컨텍스트 부담이 크게 줄어듭니다.

200:1

압축비

20만 줄 → 약 1천 템플릿 수준

98%+

토큰 절감

중복 로그 제거로 입력 토큰 대폭 감소

1회

컨텍스트 적재

한 번의 호출로 전체 로그 윤곽 파악

* 수치는 로그 중복도에 따른 개념적 예시이며, 실제 절감률은 데이터에 따라 달라집니다.

원본 ↔ 템플릿 매핑으로 추적성을 지킨다

압축은 정보 손실이 아닙니다. `template_id`가 압축 템플릿과 모든 원본 로그를 잇는 키 역할을 하여, 언제든지 원본을 추적할 수 있습니다.

원본 로그 필드

line_number

원본 라인 번호

datetime

로그 날짜/시간

system

시스템 이름

level

로그 레벨

thread / category

스레드 · 카테고리

message / raw

원본 메시지 · 전체

템플릿 매핑 필드

template_id

클러스터 ID · 매핑 키

template

압축 템플릿 (<*>)

cluster_size

동일 패턴 로그 수

← *template_id* 가 매핑 키

SQL처럼 로그를 질의한다

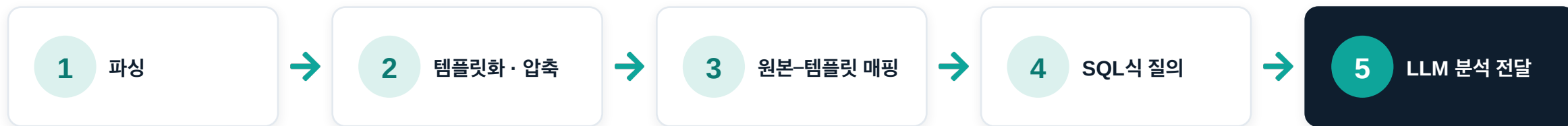
DataFrame 검색 함수는 익숙한 SQL 절(WHERE · LIKE · BETWEEN · GROUP BY)에 그대로 대응됩니다.

분석 목적	DataFrame 함수	SQL 대응
키워드 검색	<code>search_by_keyword()</code>	<code>WHERE message LIKE '%...%'</code>
필드 필터	<code>search_by_field()</code>	<code>WHERE level = 'ERROR'</code>
템플릿별 조회	<code>search_by_template_id()</code>	<code>WHERE template_id = 2</code>
시간 범위	<code>search_by_datetime_range()</code>	<code>WHERE datetime BETWEEN a AND b</code>
복합 조건	<code>search_combined()</code>	<code>WHERE ... AND ... AND ...</code>
패턴 집계	<code>build_cluster_summary_*</code>	<code>GROUP BY template</code>

```
예 search_combined(df, system="APIGateway", level="INFO", keyword="GET") ≡ SELECT * WHERE system='APIGateway' AND level='INFO' AND msg LIKE '%GET%'
```

전체 워크플로우

원본 로그를 압축·매핑·질의하고, 압축된 템플릿을 LLM에 전달하기까지의 흐름입니다.



pipeline.py

```
# 1. 파싱
entries = parse_log_text(log_text)
# 2. 템플릿화 (압축)
miner, results = extract_templates_from_entries(entries)
# 3. 원본-템플릿 매핑 DataFrame
merged_df = build_merged_dataframe(log_df, tpl_df)
# 4. SQL식 질의
errors = search_combined(merged_df, level="ERROR")
# 5. 압축 템플릿을 LLM에 전달
summary = build_cluster_summary_dataframe(merged_df)
```

운영 가능한 에이전트, 무엇이 필요한가

좋은 답을 한 번 생성하는 것을 넘어, 왜 그 답이 나왔는지 재현·검증·개선할 수 있어야 합니다.

Q1 AI Agent가 왜 시간이 지나며 목표에서 벗어나는가?

Goal Drift

Q2 왜 더 긴 컨텍스트 창이 항상 더 나은 답을 만들지 않는가?

Long Context Failure

Q3 LangGraph 에이전트에서 컨텍스트는 어디에 존재하는가?

Context Architecture

Q4 Judge Agent는 어떤 데이터를 보고 drift를 판단해야 하는가?

Evaluation

Q5 운영 가능한 에이전트를 만들려면 어떤 telemetry가 필요한가?

Observability

Agent Drift — 에이전트가 목표에서 벗어나는 현상



Goal Drift

원래 목표에서 이탈



Tool Drift

잘못된 도구 선택·반복 호출



Memory Drift

관련 없는 / 오래된 기억 사용



RAG Drift

문서 오해석 · 과도한 일반화



Evidence Drift

근거 없이 수차·원인 주장



Handoff Drift

노드 간 전달 시 의미 변형



Context Bloat

불필요한 컨텍스트 과다 누적



Context Poisoning

잘못된 정보가 State 오염

필요한 것을 정확히 넣는 4대 전략

Prompt Engineering이 “어떻게 말할까” 라면, Context Engineering은 “무엇을 보여줄까” 의 문제입니다.



Write

컨텍스트를 외부에 기록

scratchpad

memory

checkpoint



Select

필요한 컨텍스트만 선택

RAG

retrieval

tool selection



Compress

핵심만 남기고 압축

summary

compaction



Isolate

컨텍스트를 역할별 격리

subgraph

subagent

scope

컨텍스트를 State · Node · Edge로 통제한다

LangGraph는 컨텍스트가 어디에 저장되고 어떤 흐름으로 전달되는지를 명시적으로 다룰 수 있게 합니다.

State

현재 애플리케이션 snapshot · 컨텍스트 저장소

Node

state를 읽고 작업 수행 후 update 반환

Edge

다음 node 결정 · Context Flow Control

Checkpointing

state snapshot 저장 · drift 추적 가능

Store

long-term memory 저장 · thread 넘어 유지

Context Engineering 장점

- ✓ 컨텍스트를 state channel로 분리
- ✓ node별로 필요한 컨텍스트만 전달
- ✓ checkpoint로 오염 지점 추적
- ✓ subgraph로 context isolation
- ✓ validation node로 final gate 구성

무엇이 기록되는가 — 이벤트 체계

에이전트 실행의 모든 단계를 구조화된 이벤트로 남겨, Judge Agent가 검증할 수 있는 증거를 만듭니다.

실행 시작/종료

run_start
run_end
final_output

명령 스냅샷

instruction_snapshot

노드 흐름

node_start
node_end
edge_selected

ReAct 루프

react_step
observation

도구 호출

tool_start
tool_end
tool_error

LLM 호출

llm_start
llm_end
llm_error

RAG / MCP

retrieval_result
mcp_request
mcp_response

검증 결과

validation_result

최소 구현 run_start/end · instruction_snapshot · llm_start/end · tool_start/end · final_output · validation_result

어떤 Telemetry가 어떤 Drift를 잡는가

Drift 유형	필요한 이벤트	탐지 포인트
Goal drift	run_start · final_output · validation_result	목표 vs 결과 비교
Prompt drift	instruction_snapshot	instruction 변경 이력
Tool drift	react_step · tool_start/end	잘못된 도구 선택·반복
Evidence drift	tool_end · final_output · validation_result	근거 없는 수치 주장
RAG drift	retrieval_result · final_output	문서 오해석
Handoff drift	node_start/end · edge_selected	노드 간 의미 변형
Context bloat	llm_start messages · token usage	과다 토큰 사용
Context poisoning	checkpoints · node state diff · errors	오염 state 추적

Judge Agent 평가 루프

Observe → Judge → Improve — 실행을 관측하고, drift를 판정하고, 컨텍스트 정책으로 개선을 되먹임합니다.

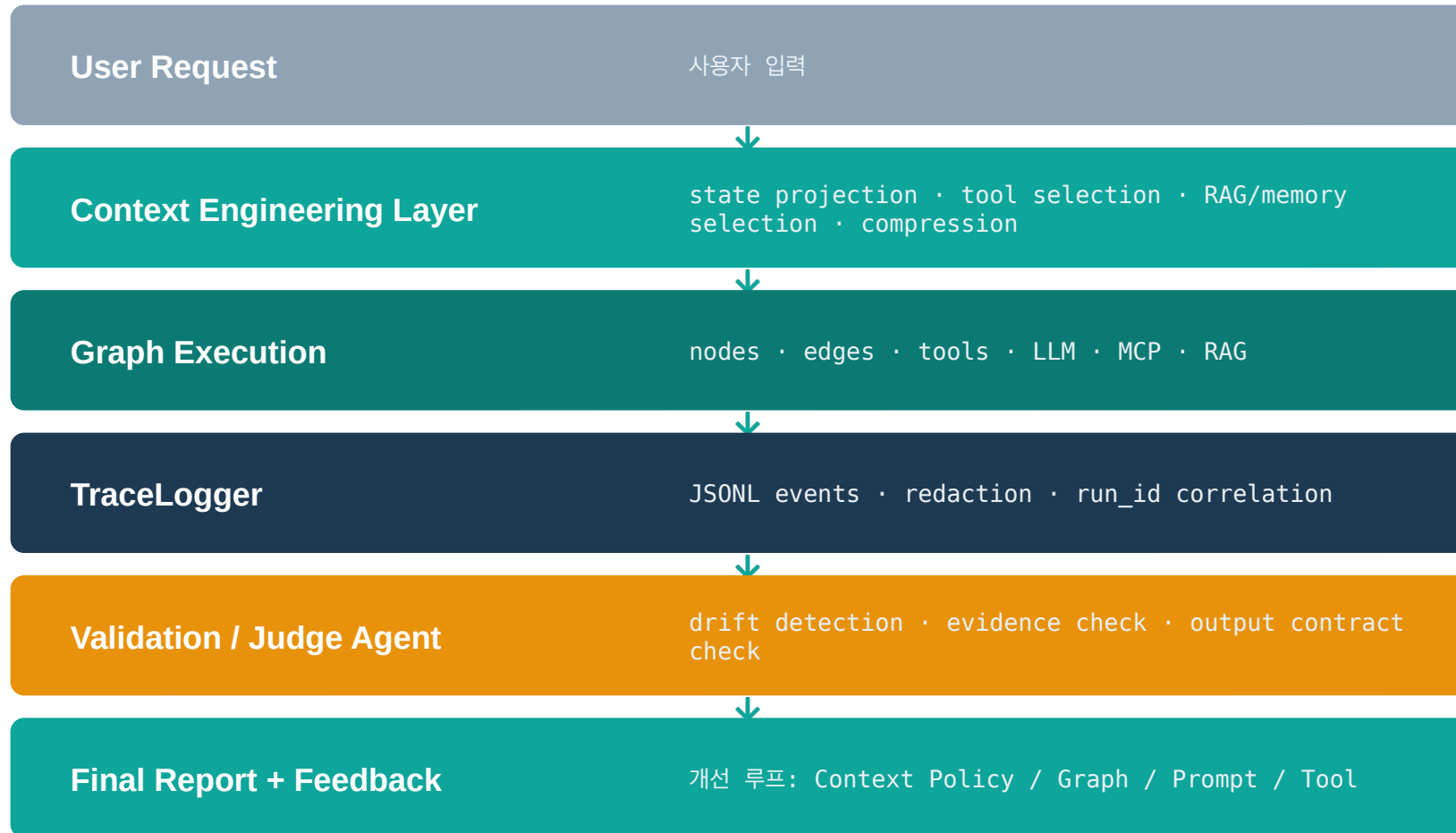


Judge Agent 평가 항목

- ✓ Drift 유형 분류
- ✓ Evidence alignment 검사
- ✓ Context bloat 측정
- ✓ Tool / RAG / Memory 사용 평가
- ✓ Validation issue 생성
- ✓ 출력 계약 위반 탐지

통합 아키텍처

Context Engineering + LangGraph + Drift Telemetry



개선 루프

Judge Agent의 검증 결과가 Context Policy · Graph · Prompt · Tool 설계로 되돌아가 시스템을 지속적으로 개선합니다.

구현 로드맵 — 4단계 계획

1단계

Trace 표준화

- TraceLogger 인터페이스 고정
- event schema 문서화
- redaction 강화
- JSONL + collector 이중화

2단계

Context 구조화

- State schema 정리
- raw / summary 분리
- evidence / hypothesis 분리
- node별 context projection

3단계

Drift Judge 강화

- trace 기반 drift rule 구현
- LLM-as-a-Judge 평가 추가
- regression fixture 구성

4단계

운영화

- dashboard & alert 구축
- prompt/tool/version 비교
- context budget 추적

SUMMARY

핵심 정리

1

로그 압축

Drain3로 반복 로그를 템플릿화해 LLM 토큰을 절감하고,
template_id로 원본 추적성을 유지한다.

2

Context Engineering

모델이 무엇을 보고 판단할지 설계하고,
Write·Select·Compress·Isolate로 컨텍스트를 통제한다.

3

Drift Telemetry

에이전트가 실제로 어떤 context와 도구를 사용했는지 기록하는 증거
계층을 만든다.

4

Judge Agent

trace를 기반으로 drift · evidence alignment · 출력 계약 위반을
평가하는 검증자를 둔다.

“운영 가능한 시스템은 좋은 답을 한 번 생성하는 게 아니라, 왜 그 답이 나왔는지 재현·검증·개선할 수 있는 시스템이다.”